Cite as: Rintanen, A.: Coupling OpenFOAM with Cantera for advanced diffusion modeling. In Proceedings of CFD with OpenSource Software, 2024, Edited by Nilsson. H.,
http://dx.doi.org/10.17196/OS_CFD#YEAR_2024

CFD with OpenSource software

A course at Chalmers University of Technology
Taught by Håkan Nilsson

# Coupling OpenFOAM with Cantera for advanced diffusion modeling

Developed for OpenFOAM-v12
Requires: Cantera

*Author:*
Aleksi Rintanen
Aalto University
aleksi.rintanen@aalto.fi

*Peer reviewed by:*
FirstName1 LastName1
FirstName2 LastName2

Licensed under CC-BY-NC-SA, https://creativecommons.org/licenses/

Disclaimer: This is a student project work, done as part of a course where OpenFOAM and some other OpenSource software are introduced to the students. Any reader should be aware that it might not be free of errors. Still, it might be useful for someone who would like to learn some details similar to the ones presented in the report and in the accompanying files. The material has gone through a review process. The role of the reviewer is to go through the tutorial and make sure that it works, that it is possible to follow, and to some extent correct the writing. The reviewer has no responsibility for the contents.

December 21, 2024

# Learning outcomes

The main requirements of a tutorial in the course is that it should teach the four points: How to use it, The theory of it, How it is implemented, and How to modify it. Therefore the list of learning outcomes is organized with those headers.

The reader will learn:

**How to use it:**

- How to use and select different diffusion models

**The theory of it:**

- The theory of various diffusion models
- Listing of the currently available models in OpenFOAM and the equations they implement

**How it is implemented:**

- How different diffusion models are implemented
- What is the structure of the nested runtime tables and how the models are added to them

**How to modify it:**

- Create a new diffusion model
- Couple Cantera with OpenFOAM
- Calculate diffusion coefficients and thermal diffusivities in Cantera

# Prerequisites

- How to run standard tutorials like damBreak tutorial
- Fundamentals of diffusion phenomena in gases
- Basic knowledge on CFD and compressible flow with multiple components
- How to customize a generic OpenFOAM class

# Contents


**1 Introduction** **5**

**2 Theoretical background** **6**
2.1 Governing equations . . . . . 6
2.2 Fick's law of diffusion . . . . . 6
2.3 Multicomponent diffusion model . . . . . 7
2.3.1 Thermal diffusion . . . . . 7

**3 Thermophysical transport models in OpenFOAM** **8**
3.1 Similarity to momentum transport models . . . . . 8
3.2 Available models . . . . . 9
3.3 Structure of the models . . . . . 10

**4 Implementation** **13**
4.1 Coupling Cantera with OpenFOAM . . . . . 13
4.1.1 Option 1: installing from a package repository . . . . . 13
4.1.2 Option 2: compiling from source code . . . . . 13
4.1.3 Using Cantera from OpenFOAM . . . . . 14
4.2 Creating a new thermophysical transport model . . . . . 14
4.3 Modifications to the code . . . . . 14
4.3.1 Correction velocity approach . . . . . 15
4.3.2 Boundary conditions for thermal diffusion coefficients . . . . . 16

**5 Model validation and tutorials** **17**
5.1 Validation . . . . . 17
5.2 Tutorial . . . . . 18

**6 Conclusions and future work** **19**

# Nomenclature

**Acronyms**

| | |
|---|---|
| CFD | Computational Fluid Dynamics |

**English symbols**

| | | |
|---|---|---|
| $\mathbf{j}$ | Diffusive mass flux vector | $\mathrm{kg/m^2/s}$ |
| $\mathbf{q}$ | Diffusive heat flux vector | $\mathrm{J/m^2/s}$ |
| $\mathbf{u}$ | Velocity vector | $\mathrm{m/s}$ |
| $\mathbf{V}$ | Diffusion velocity vector | $\mathrm{m/s}$ |
| $\mathcal{D}_{ij}$ | Binary diffusion coefficient | $\mathrm{m^2/s}$ |
| $\overline{W}$ | Average molar mass | $\mathrm{kg/kmol}$ |
| $c_p$ | Heat capacity at constant pressure | $\mathrm{J/(K \cdot kg)}$ |
| $D_k$ | Mass diffusion coefficient | $\mathrm{m^2/s}$ |
| $D_k^T$ | Thermal diffusion coefficient | $\mathrm{kg/(m \cdot s)}$ |
| $D_{ij}$ | Ordinary multicomponent diffusion coefficient | $\mathrm{m^2/s}$ |
| $h$ | Specific enthalpy | $\mathrm{J/kg}$ |
| $h_s$ | Specific sensible enthalpy | $\mathrm{J/kg}$ |
| $K$ | Specific kinetic energy | $\mathrm{m^2/s^2}$ |
| $Le$ | Lewis number | |
| $p$ | Pressure | $\mathrm{Pa}$ |
| $Pr$ | Prandtl number | |
| $Pr_t$ | Turbulent Prandtl number | |
| $Sc$ | Schmidt number | |
| $Sc_t$ | Turbulent Schmidt number | |
| $T$ | Temperature | $\mathrm{K}$ |
| $t$ | time | $\mathrm{s}$ |
| $W$ | Molar mass | $\mathrm{kg/kmol}$ |
| $X$ | Molar fraction | |
| $Y$ | Mass fraction | |

**Greek symbols**

| | | |
|---|---|---|
| $\lambda$ | Thermal conductivity | $\mathrm{W \cdot m/K}$ |
| $\nu$ | Fluid kinematic viscosity | $\mathrm{m^2/s}$ |
| $\nu_t$ | Turbulent viscosity | $\mathrm{m^2/s}$ |
| $\omega$ | Reaction source term | $\mathrm{kg/m^3/s}$ |
| $\omega_T$ | Heat release rate | $\mathrm{W/m^3}$ |
| $\rho$ | Fluid density | $\mathrm{kg/m^3}$ |

# Chapter 1

# Introduction

Recent increased interest in hydrogen combustion has created a demand to develop tools to model diffusion effects more accurately. The usual assumption that species diffusivities are the same and equal to thermal diffusivity does not hold well for hydrogen flames [1]. More advanced diffusion models were introduced in OpenFOAM-v9, but they do not include an accurate representation of thermal diffusion, also known as the Soret effect, where a diffusion flux is formed due to a temperature gradient. To be more precise, user can add the Soret effect by providing an user-defined `function2` object, where the thermal diffusion coefficient could be calculated from e.g. polynomial expressions of temperature and pressure. However, the usual models used in literature are much more complex than simple polynomial expressions of temperature and pressure. Also, there does not exist any overviews of the implemented diffusion models in OpenFOAM and therefore a short summary of the models and their equations will be given.

Cantera is an open-source software for simulating chemically reacting system in 0D or 1D configurations [2]. Cantera has been coupled with OpenFOAM in multiple reactive flow studies and used for calculating chemical source terms and thermodynamical properties. Cantera can also evaluate the needed thermophyscial properties related to accurate diffusion modeling. This has been done e.g. Thorsten et al. [3], and while the idea is not new, there are not any openly published codes or tutorials how it can be done to the author's knowledge.

# Chapter 2

# Theoretical background

## 2.1 Governing equations

The governing equations in `multicomponentFluid` solver module include the complete Navier-Stokes equations (density, momentum, and sensible enthalpy) and the species transport equations. Energy equation can be solved for multiple solution variables in OpenFOAM, but here we consider sensible enthalpy $h_s$ for simplicity. Equations for species transport and enthalpy are given as

$$\frac{\partial}{\partial t}(\rho Y_k) + \nabla \cdot (\rho \mathbf{u} Y_k) = -\nabla \cdot \mathbf{j}_k + \dot{\omega}_k \tag{2.1}$$

and

$$\frac{\partial}{\partial t}(\rho h_s) + \nabla \cdot (\rho \mathbf{u} h_s) + \frac{\partial}{\partial t}(\rho K) + \nabla \cdot (\rho \mathbf{u} K) = \frac{\partial p}{\partial t} - \nabla \cdot \mathbf{q} + \dot{\omega}_T, \tag{2.2}$$

where evaluation of species diffusive flux $\mathbf{j}$ and heat flux $\mathbf{q}$ depends on the used diffusion model. The heat flux is evaluated assuming Fourier's law:

$$\mathbf{q} = -\lambda \nabla T + \sum_{k=1}^{N} h_{s,k} \mathbf{j}_k, \tag{2.3}$$

where $h_{s,k}$ is the sensible enthalpy of specie $k$. In OpenFOAM term thermophysical transport model is used to describe models that evaluate fluxes $\mathbf{j}_k$ and $\mathbf{q}$.

## 2.2 Fick's law of diffusion

Here, derivation and notation for diffusion models follows Kee's book [4]. Fick's law states that diffusive mass flux depends linearly on the concentration gradient, such that the flux is towards low-concentration areas and the flux can be written as

$$\mathbf{j}_k = -\rho \frac{W_k}{\overline{W}} D_k \nabla X_k, \tag{2.4}$$

where $W_k$ and $\overline{W}$ are the molar weight of specie $k$ and the average molar weight respectively, and $D_k$ is diffusion coefficient for specie $k$. The diffusion coefficient for a two-component system can be measured, but for a general multicomponent mixture, the evaluation of diffusion coefficient is more complex. In mixture-averaged model, the diffusion coefficients are estimated from the Maxwell–Stefan diffusion model using simplifying assumptions.

The diffusion flux can be rewritten in terms of mass fraction gradient as

$$\mathbf{j}_k = -\rho D'_k \nabla Y_k, \tag{2.5}$$

which is in our case more convenient, since the equations for species are solved with respect to mass fractions in OpenFOAM. The mixture-averaged diffusion coefficients $D'_k$ are then given as:

$$D'_k = \left( \sum_{j \neq k} \frac{X_j}{\mathcal{D}_{jk}} + \frac{X_k}{1 - Y_k} \sum_{j \neq k} \frac{Y_j}{\mathcal{D}_{jk}} \right)^{-1}, \tag{2.6}$$

where $\mathcal{D}_{jk}$ is the binary diffusion coefficient between species $j$ and $k$. A common simplification is to assume unity Lewis number $Le_k = \alpha / D_k = 1$ for all the species, where mass diffusivity $D_k$ equals thermal diffusivity $\alpha$. This simplifies the equations even further and now the heat flux $\mathbf{q}$ can be simplified using the relation $dh_{s,k} = c_{p,k}\, dT$ to:

$$\mathbf{q} = -\lambda \nabla T + \sum_{k=1}^{N} h_{s,k} \mathbf{j}_k = -\frac{\lambda}{c_p} \nabla h_s. \tag{2.7}$$

## 2.3 Multicomponent diffusion model

More general and accurate model is given by the multicomponent diffusion model, which defined as:

$$\mathbf{j}_k = \rho \frac{W_k}{\overline{W}} \sum_{j=1}^{N} W_j D_{kj} \left( \nabla X_j + (X_j - Y_j) \frac{1}{p} \nabla p \right) - D_k^T \frac{1}{T} \nabla T, \tag{2.8}$$

where $D_{jk}$ is the ordinary multicomponent diffusion coefficient and $D_k^T$ is thermal diffusion coefficient. Notice that these ordinary multicomponent diffusion coefficient are different from the binary diffusion coefficients and they need to be solved from a matrix equation, which is constructed based on the kinetic theory of gases. The diagonal of the ordinary multicomponent diffusion coefficient matrix $D_{jk}$ is zero, which can pose some numerical challenges. An alternative for the multicomponent model is the Maxwell-Stefan model, where diffusion velocities are related to the field gradients as:

$$\nabla X_k = \sum_{j=1}^{N} \frac{X_k X_j}{\mathcal{D}_{kj}} (\mathbf{V}_j - \mathbf{V}_k) + (Y_k - X_k) \left( \frac{\nabla p}{p} \right) + \sum_{j=1}^{N} \frac{X_k X_j}{\rho \mathcal{D}_{kj}} \left( \frac{D_j^T}{Y_j} - \frac{D_k^T}{Y_k} \right) \frac{\nabla T}{T}, \tag{2.9}$$

where $\mathbf{V}_k$ is the diffusion velocity for species $k$. The diffusion due to pressure gradient is often neglegted.

### 2.3.1 Thermal diffusion

Thermal diffusion, also known as Soret effect, is diffusion due to temperature gradient, represented by the last term in the multicomponent model in equation (2.8). The evaluation of the thermal diffusion coefficients $D_k^T$ in the multicomponent model requires solving a linear system of size $3N \times 3N$, where $N$ is the number of species. In contrast to the mixture-averaged model, which does not require linear solve or multicomponent/Maxwell-Stefan model which requires solving of $N \times N$ system, this is computationally much heavier.

The thermal diffusion term can also be included in the mixture-averaged model. There are a few lightweight but less accurate models that can be used with the mixture-averaged model [5]. Here in this implementation, we are including the full thermal diffusion model.

# Chapter 3

# Thermophysical transport models in OpenFOAM

## 3.1 Similarity to momentum transport models

Thermophysical transport models were introduced in OpenFOAM-v9, and until now those are only present in the foundation versions. If the reader is familiar with the concept of momentum transport models in OpenFOAM, the idea of thermophysical transport model is the same, but applied for the species and energy equations providing the diffusive terms to transport equations. The benefit of thermophysical/momentum transport models is that they are solver independent modules that can be used interchangeably among different solvers similarly as turbulence models are used.

User defines the model and model coefficients in the `thermophysicalTransportDict` dictionary file in the constant directory. When a laminar turbulence model is used, the model name is looked up under the `laminar` subdictionary. For RAS/LES simulations, their corresponding subdictionaries are used. An example is given in a listing below.

Listing 3.1: An example of thermophysicalTransportDict file

```
1
2  /*--------------------------------*- C++ -*----------------------------------*\
3    =========                 |
4    \\      /  F ield         | OpenFOAM: The Open Source CFD Toolbox
5     \\    /   O peration     | Website:  https://openfoam.org
6      \\  /    A nd           | Version:  12
7       \\/     M anipulation  |
8  \*---------------------------------------------------------------------------*/
9  FoamFile
10 {
11     version         2;
12     format          ascii;
13     class           dictionary;
14     location        "constant";
15     object          thermophysicalTransport;
16 }
17 // * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //
18
19 laminar
20 {
21     model           unityLewisFourier;
22 }
23
24
25 // ************************************************************************* //
```

## 3.2 Available models

There are several base instances for thermophysical transport models similarly as there are for momentum transport models (icompressible, compressible etc.), which are used by different solvers. Each of these instances have three base models in their runtime tables: `laminar`, `RAS`, and `LES`. Each base model has its own selector and runtime table for the actual models. For the `fluidMulticomponentThermo`, which is used by the reactive solver `multicomponentFluid`, the available models are shown in the file below.

Listing 3.2: fluidMulticomponentThermophysicalTransportModels.H

```
#include "fluidMulticomponentThermophysicalTransportModels.H"

// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //

makeThermophysicalTransportModelTables
(
    ThermophysicalTransportModel,
    compressibleMomentumTransportModel,
    fluidMulticomponentThermo
);


// ------------------------------------------------------------------------- //
// Laminar models
// ------------------------------------------------------------------------- //

#include "Fourier.H"
makeLaminarThermophysicalTransportModel(Fourier);

#include "unityLewisFourier.H"
makeLaminarThermophysicalTransportModel(unityLewisFourier);

#include "FickianFourier.H"
makeLaminarThermophysicalTransportModel(FickianFourier);

#include "MaxwellStefanFourier.H"
makeLaminarThermophysicalTransportModel(MaxwellStefanFourier);


// ------------------------------------------------------------------------- //
// RAS models
// ------------------------------------------------------------------------- //

#include "eddyDiffusivity.H"
makeRASLESThermophysicalTransportModel(RAS, eddyDiffusivity);

#include "unityLewisEddyDiffusivity.H"
makeRASLESThermophysicalTransportModel(RAS, unityLewisEddyDiffusivity);

#include "nonUnityLewisEddyDiffusivity.H"
makeRASLESThermophysicalTransportModel(RAS, nonUnityLewisEddyDiffusivity);

#include "FickianEddyDiffusivity.H"
makeRASLESThermophysicalTransportModel(RAS, FickianEddyDiffusivity);


// ------------------------------------------------------------------------- //
// LES models
// ------------------------------------------------------------------------- //

#include "eddyDiffusivity.H"
makeRASLESThermophysicalTransportModel(LES, eddyDiffusivity);

#include "unityLewisEddyDiffusivity.H"
makeRASLESThermophysicalTransportModel(LES, unityLewisEddyDiffusivity);

```

```
57 #include "nonUnityLewisEddyDiffusivity.H"
58 makeRASLESThermophysicalTransportModel(LES, nonUnityLewisEddyDiffusivity);
```

Macro `makeThermophysicalTransportModelTables` creates the runtime tables for the base models. Macros `makeLaminarThermophysicalTransportModel` and `makeRASLESThermophysicalTransportModel` add the actual models to the corresponding runtime tables.

The names are constructed from two parts: suffix "Fourier" means that the heat flux is propotional to the temperature gradient. "Eddydiffusivity" is the LES/RAS counterpart and here the turbulent closure is modelled by using eddy diffusivity concept, where additional diffusion is introduced by assuming a relation to the turbulent viscosity $\nu_t$ as:

$$\begin{aligned} \alpha_{\text{eff}} &= \alpha + \frac{\nu_t}{Pr_t} \\ D_{k\text{eff}} &= D_k + \frac{\nu_t}{Sc_t}, \end{aligned} \tag{3.1}$$

where the turbulent viscosity is scaled by turbelent Prandtl number $Pr_t$ and Schimdt number $Sc_t$. The models and the definitions of diffusive fluxes are listed in Table 3.1. Since the energy equation is not solved for temperature but for enthalpy or internal energy, adding the temperature gradient explicitly would affect greatly on the stability of the solver. OpenFOAM uses a trick where an implicit correction term is added to the equations, which vanishes when the solution converges. More details of the approach are available in Greenshields' book [6].

Table 3.1: Available transport models and implementation of diffusive fluxes

| Model | $\mathbf{j}_k$ | $\mathbf{q}$ | Type |
|---|---|---|---|
| `Fourier`[1] | - | $-\lambda\nabla T$ | laminar |
| `unityLewisFourier` | $-\frac{\lambda}{c_p}\nabla Y_k$ | $-\frac{\lambda}{c_p}\nabla h_s$ | laminar |
| `FickianFourier` | $-D_k\nabla Y_k$ | $-\frac{\lambda}{c_p}\nabla T + \sum_k h_{sk}\mathbf{j}_k$ | laminar |
| `MaxwellStefanFourier` | See eq.(2.9) ($\nabla p = 0$) | $-\lambda\nabla T + \sum_k h_{sk}\mathbf{j}_k$ | laminar |
| `eddyDiffusivity`[1] | - | $-(\lambda + \frac{\nu_t}{Pr_t})\nabla T$ | RAS/LES |
| `unityLewisEddyDiffusivity` | $-(\frac{\lambda}{c_p} + \frac{\nu_t}{Pr_t})\nabla Y_k$ | $-(\frac{\lambda}{c_p} + \frac{\nu_t}{Pr_t})\nabla h_s$ | RAS/LES |
| `nonUnityLewisEddyDiffusivity` | $-(\frac{\lambda}{c_p} + \frac{\nu_t}{Sc_t})\nabla Y_k$ | $-(\frac{\lambda}{c_p} + \frac{\nu_t}{Pr_t})\nabla h_s$ | RAS/LES |
| `FickianEddyDiffusivity` | $-(D_k + \frac{\nu_t}{Sc_t})\nabla Y_k$ | $-(\frac{\lambda}{c_p} + \frac{\nu_t}{Pr_t})\nabla T + \sum_k h_{sk}\mathbf{j}_k$ | RAS |

The diffusion coefficient for the Maxwell-Stefan and mixture-averaged `Fickian` based models can either be given as a list of key-value pairs or as a user-defined function of pressure and temperature. This function is defined as OpenFOAM's `function2` object, which the user needs to write and compile.

Soret effect is also included in both models similarly as the diffusion coefficients via `function2` interface. However, it is highly inaccurate to assume that this would be only a function of temperature and pressure.

## 3.3 Structure of the models

The two most important class members that are re-implemented in the transport models are functions `divq(volScalarField& he)` and `divj(volScalarField& Yi)`, of which are returning the divergence of diffusive fluxes $\mathbf{j}_k$ and $\mathbf{q}$, respectively. These methods are called when the specie and energy equation are constructed in the solver modules. The models can be found under the folder `src/ThermophysicalTransportModels/fluid`. The models are templated based on the thermo and turbulence models and the instances are created based on the macros described in the previous section. The class interface for the `unityLewisFourier` class is given in a listing below. Members `q()` and `j(const volScalarField& Yi)` give the volumetric fluxes of $\mathbf{j}_k$ and $\mathbf{q}$. `predict()` is a new

[1] Supports single component systems only

name for the earlier function `correct()` in thermophysical/momentum transport models, which updates the models.

Listing 3.3: unityLewisFourier.H

```
template<class BasicThermophysicalTransportModel>
class unityLewisFourier
:
    public laminarThermophysicalTransportModel
    <
        BasicThermophysicalTransportModel
    >
{

public:

    typedef typename BasicThermophysicalTransportModel::alphaField
        alphaField;

    typedef typename BasicThermophysicalTransportModel::momentumTransportModel
        momentumTransportModel;

    typedef typename BasicThermophysicalTransportModel::thermoModel
        thermoModel;


    //- Runtime type information
    TypeName("unityLewisFourier");


    // Constructors

        //- Construct from a momentum transport model and a thermo model
        unityLewisFourier
        (
            const momentumTransportModel& momentumTransport,
            const thermoModel& thermo
        );

        //- Construct from a type name, a momentum transport model and a thermo
        //  model
        unityLewisFourier
        (
            const word& type,
            const momentumTransportModel& momentumTransport,
            const thermoModel& thermo
        );


    //- Destructor
    virtual ~unityLewisFourier()
    {}


    // Member Functions

        //- Const access to the coefficients dictionary
        virtual const dictionary& coeffDict() const;

        //- Read thermophysicalTransport dictionary
        virtual bool read();

        //- Effective mass diffusion coefficient
        //  for a given specie mass-fraction [kg/m/s]
        virtual tmp<volScalarField> DEff(const volScalarField& Yi) const
        {
            return volScalarField::New
            (
```

```
64                  "DEff",
65                  this->thermo().kappa()/this->thermo().Cp()
66              );
67          }
68
69          //- Effective mass diffusion coefficient
70          //  for a given specie mass-fraction for patch [kg/m/s]
71          virtual tmp<scalarField> DEff
72          (
73              const volScalarField& Yi,
74              const label patchi
75          ) const
76          {
77              return
78                  this->thermo().kappa().boundaryField()[patchi]
79                 /this->thermo().Cp().boundaryField()[patchi];
80          }
81
82          //- Return the heat flux [W/m^2]
83          virtual tmp<surfaceScalarField> q() const;
84
85          //- Return the source term for the energy equation
86          virtual tmp<fvScalarMatrix> divq(volScalarField& he) const;
87
88          //- Return the specie flux for the given specie mass-fraction [kg/m^2/s]
89          virtual tmp<surfaceScalarField> j(const volScalarField& Yi) const;
90
91          //- Return the source term for the given specie mass-fraction equation
92          virtual tmp<fvScalarMatrix> divj(volScalarField& Yi) const;
93
94          //- Correct the unityLewisFourier viscosity
95          virtual void predict();
96  };
```

# Chapter 4

# Implementation

## 4.1 Coupling Cantera with OpenFOAM

First c++ version of Cantera needs to be installed. Cantera can be installed from pre-compiled binaries or can be compiled from the source code.

### 4.1.1 Option 1: installing from a package repository

Cantera can be installed from binaries for certain Linux distros. Here instructions are given for Ubuntu 22.04, but instructions for other distros can be found from Cantera's homepage. Start by adding Cantera's Personal Package Archive (PPA) to sources:

```
sudo apt install software-properties-common
sudo apt-add-repository ppa:cantera-team/cantera
```

Cantera (C++ version) with headers can then be installed with:

```
sudo apt install libcantera-dev
```

Settings for compilation and linking of OpenFOAM programs/libraries utilizing Cantera can be automated using `pkg-config` tool, where `pkg-config --cflags cantera` gives the compilation flags and `pkg-config --libs cantera` linking flags. An example of OpenFOAM's `options` file is given in below. The needed `yaml-cpp`-library, which is shipped along the Cantera installation, is not linked by default and must be given explicitly.

Listing 4.1: Example of OpenFOAM's options file

```
EXE_INC = \
     $(shell pkg-config --cflags cantera)

LIB_LIBS = \
     -lyaml-cpp \
      $(shell pkg-config --libs cantera)
```

### 4.1.2 Option 2: compiling from source code

The project code package has a script `makeCantera` that will clone Cantera 3.0.0 github repository and compile the code. Cantera is built using Scons built-system. Installing Cantera to the system (`scons install`) is not mandatory, but then the compile flags, include paths, and linking flags needs to be provided manually and not by `pkg-config`. Notice that Cantera requires c++17 standard. Cantera can be linked either statically (`libcantera.a`) or dynamically (`libcantera_shared.so`).

### 4.1.3 Using Cantera from OpenFOAM

The basic Cantera object is a solution object that can be created from a chemical mechanism file `mech` as

```
Cantera::shared_ptr<Cantera::Solution> solution = Cantera::newSolution(mech);
```

Since the species can have different order in Cantera, an indexing list `List<label> speciesIndex_` is used as a mapping between OpenFOAM and Cantera species indicies. Cantera uses the standard std-containers and mapping can be obtained looping over the specie names:

Listing 4.2: Constructing species mapping between Cantera and OpenFOAM

```
const std::vector<std::string> & speciesNames = solution_->thermo()->speciesNames();
forAll(Y, i)
{
    const std::vector<std::string>::const_iterator iter =
        std::find(speciesNames.begin(), speciesNames.end(), Y[i].name());
    if(iter == speciesNames.end())
    {
        FatalErrorInFunction
            << "Specie " << Y[i].name() << " not found in Cantera mechanism."
            << exit(FatalError);
    }
    speciesIndex_[i] = std::distance(speciesNames.begin(), iter);
}
```

## 4.2 Creating a new thermophysical transport model

A new transport model is created by cloning and renaming `FickianFourier` class into `CanteraMixtureAveraged`. The previously presented `fluidMulticomponentThermophysicalTransportModels.H` is also cloned with its `Make` directory but the content is modified to only contain an instance of the `CanteraMixtureAveraged` class:

Listing 4.3: fluidMulticomponentThermophysicalTransportModels.H

```
#include "fluidMulticomponentThermophysicalTransportModels.H"

#include "CanteraMixtureAveraged.H"
makeLaminarThermophysicalTransportModel(CanteraMixtureAveraged);
```

This will now compile an instance of `CanteraMixtureAveraged` class templated on the desired momentumtransport and themophysical models ja insert it to the runtime table.

## 4.3 Modifications to the code

All the code related to calculation of diffusion coefficients is replaced. Also, a boolean flag `soretEnabled_` is added to indicate whether to include the computations for the Soret effect. Diffusion coefficients are evaluated using Cantera's `getMixDiffCoeffs` method, which evaluates the coefficients based on equation (2.6). Thermal diffusion coefficients are calculated using method `getThermalDiffCoeffs`. First, the gas mixture is set to desired properties, and then the method is called, and this procedure is repeated for all the cells in the internal field:

Listing 4.4: Evaluation of diffusion coefficients

```
Cantera::shared_ptr<Cantera::ThermoPhase> gas = solution_->thermo();
scalarField Ys(Y.size());
scalarField Dms(Y.size());
scalarField DTs(Y.size());
scalar epsilon = 1-Y.size()*SMALL;
forAll(T,celli) {
```

```
    forAll(Y,i) {
        if(Y[i][celli] > epsilon)
        {
            Ys[speciesIndex_[i]] = epsilon;
        }
        else {
            Ys[speciesIndex_[i]] = Y[i][celli] + SMALL;
        }
    }
    gas->setState_TPY(T[celli], p[celli], Ys.begin());
    solution_->transport()->getMixDiffCoeffs(Dms.begin());
    forAll(Y,i) {
        Dm_[i][celli] = Dms[speciesIndex_[i]];
    }
    if(soretEnabled_) {
        solution_->transport()->getThermalDiffCoeffs(DTs.begin());
        forAll(Y,i) {
            DT_[i][celli] = DTs[speciesIndex_[i]];
        }
   }
}
```

Since the equation (2.6) is undefined at $Y_k = 1$ and the denominator flips its sign when $Y_k > 1$, a small value is added to the all the species and $Y_k$ is restricted such that $\max Y_k < 1$ to prevent numerical problems in single component regions. A similar loop is also performed for the boundary fields.

### 4.3.1 Correction velocity approach

Summing together the transport equations for species should lead to the continuity equation, which is not satisfied by the mixture-averaged model. In other words, mixture-averaged model does not conserve mass directly, since the diffusive mass fluxes do not cancel out completely. There are different approaches how to deal with the problem and the current OpenFOAM's mixture-averaged model uses an approach were all the errors in mass fluxes are dumped into inert species. According to the author's own tests, this works fine when the gas is diluted to the inert specie everywhere, but if there is a separate inlet for the fuel, this may lead to numerical instabilities. A more accurate approach is the correction velocity approach, where the imbalance is introduced into all the species proportional to their mass fraction gradients [4]. This can be written for fluxes as

$$\mathbf{j}'_k = \mathbf{j}_k - Y_k \sum_{j=1}^{N} \mathbf{j}_k \tag{4.1}$$

Now $\sum_{k=1}^{N} \mathbf{j}'_k$ is always zero. This is implemented as a new member function and called in `predict()` function:

Listing 4.5: Calculation of the correction flux

```
template<class laminarThermophysicalTransportModel>
void CanteraMixtureAveraged<laminarThermophysicalTransportModel>::correctJc() const
{

   const PtrList<volScalarField>& Y = this->thermo().Y();
   const volScalarField& T = this->thermo().T();
   surfaceScalarField gradLogT = fvc::snGrad(T)/fvc::interpolate(T);
   Jc_ *= scalar(0);
   forAll(Y, i)
   {
        Jc_ -= fvc::interpolate(this->alpha()*this->DEff(Y[i]))*fvc::snGrad(Y[i])
          + fvc::interpolate(DT_[i])*gradLogT;
   }

}
```

The correction flux is added to the species flux implicitly and the divergence of $\mathbf{j}_k$ is then given as

```
return unityLewisFourier<laminarThermophysicalTransportModel>::divj(Yi)
           - fvc::div
             (
                 fvc::interpolate
                 (
                   DT_[this->thermo().specieIndex(Yi)]
                 )
                *fvc::snGrad(T)/fvc::interpolate(T)
                *T.mesh().magSf()
             )
            - fvm::div(Jc_*T.mesh().magSf(),Yi,"div(phi,Yi_h)");
```

### 4.3.2 Boundary conditions for thermal diffusion coefficients

The usual convention is to apply the zero gradient boundary condition for species at walls. Since the ordinary diffusion flux is propotional to the gradient of mass fractions, there is no diffusion flux through the walls. However, in the case of thermal diffusion terms, there can be a temperature gradient at the wall, which will lead to an undesired flux through the boundary if not handled correctly. Here we use an approach, where the thermal diffusion coefficient is not calculated at walls, setting it to zero to prevent undesired fluxes through walls.

Listing 4.6: Forcing thermal diffusion coefficient to zero at walls

```
const polyBoundaryMesh& pbm = this->thermo().T().mesh().boundaryMesh();
if(soretEnabled_ && !isA<wallPolyPatch>(pbm[patchi])) {
    solution_->transport()->getThermalDiffCoeffs(DTs.begin());
    forAll(Y,i) {
        DT_[i][celli] = DTs[speciesIndex_[i]];
    }
}
```

# Chapter 5

# Model validation and tutorials

## 5.1 Validation

Cantera can solve sterady-state 1D freely propagating flames, which can be used to validate the implemented transport models. Cantera have three different transport models implemented: unity-Lewis, mixture-averaged and multicomponent, of which the last one can have the Soret effect enabled.

First, in Fig. 5.1 a 1D freely propagating lean hydrogen flame with equivalence ratio 0.5 is solved in ambient conditions using OpenFOAM's built-in unity-Lewis model and the newly developed mixture-averaged and the solution is compared to solutions generated with Cantera. The same uniformly spaced grid is used in both software. The profiles agree very well and at the same time shows the importance of accurate diffusion modeling in case of hydrogen flames. The solution from unity-Lewis model differs a lot from the mixture-averaged solution.

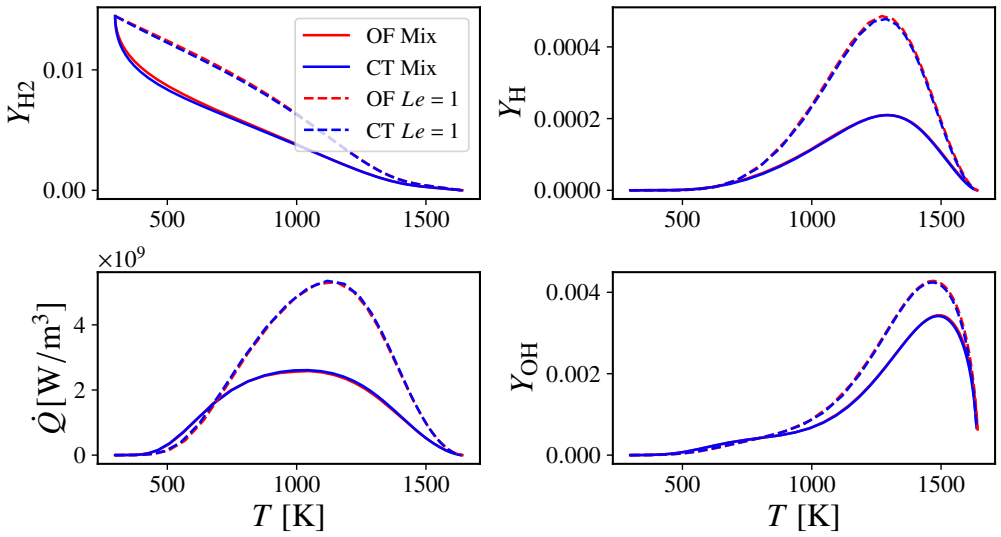



Figure 5.1: Comparison of Cantera and OpenFOAM solutions for 1D freely propagating flame using unity-Lewis and mixture-averaged transport.

Second, a comparison is made to validate the thermal diffusion model. Since Cantera does not allow using the combination of mixture-averaged transport and Soret effect, the multicomponent transport model is used. The reported differences of multicomponent and mixture-averaged diffusion model are small in the literature, and therefore the results should be comparable [4]. Fig. 5.2 shows

how the solutions from OpenFOAM and Cantera also agree well when considering thermal diffusion.

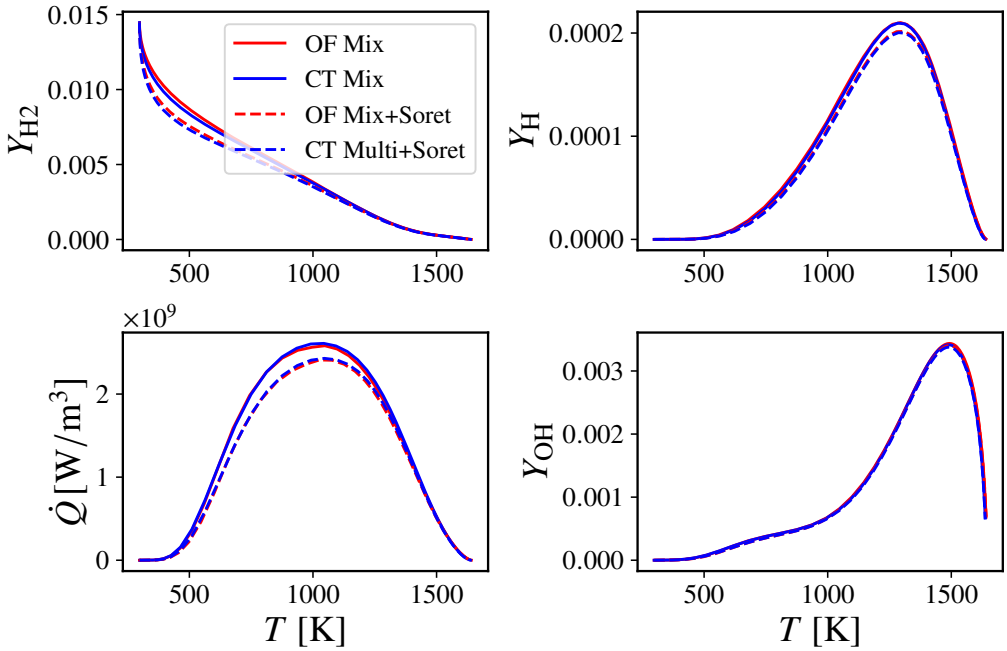



Figure 5.2: Comparison of Cantera and OpenFOAM solutions for 1D freely propagating flame with Soret effect.

## 5.2 Tutorial

`counterFlowFlame2D_GRI` is a tutorial shipped along the basic OpenFOAM installation. It simulates a 2D counterflow flame using GRI 3.0 chemical mechanism with unity Lewis number transport. A bash script and a dictionary file is given along the source code of this project, which will copy and run the basic tutorial using this newly developed `CanteraMixtureAveraged` transport model with Soret effect.

# Chapter 6

# Conclusions and future work

In the present work, Cantera software has been coupled with OpenFOAM, and a new mixture-averaged diffusion model with Soret effect was developed, where both mass and thermal diffusion coefficients were calculated inside Cantera. Additionally more robust correction velocity approach was deployed to balance the imbalance in mass fluxes. An overview and implementation of the existing thermophysical transport models in OpenFOAM were given, which at least to the author's knowledge, are not listed openly anywhere.

The model was validated by comparing the results between OpenFOAM and Cantera for one-dimensional freely propagating flames, which Cantera can solve. Good agreement in the results was obtained. As a future work, more lightweight and less complex thermal diffusion models could be developed for OpenFOAM, which would not require any linear solves.

# Bibliography


[1] H. Pitsch, "The transition to sustainable combustion: Hydrogen- and carbon-based future fuels and methods for dealing with their challenges," *Proceedings of the Combustion Institute*, vol. 40, no. 1, p. 105638, 2024. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1540748924004462

[2] C. Developers, "Community," Feb. 2024. [Online]. Available: https://cantera.org/community.html

[3] T. Zirwes, F. Zhang, P. Habisreuther, J. A. Denev, H. Bockhorn, and D. Trimis, "Implementation and validation of a computationally efficient dns solver for reacting flows in openfoam," in *14th World Congress on Computational Mechanics*. Virtual Congress, 2021.

[4] R. J. Kee, M. E. Coltrin, and P. Glarborg, *Chemically Reacting Flow: Theory and Practice*, 1st ed. Wiley, Feb. 2003. [Online]. Available: https://onlinelibrary.wiley.com/doi/book/10.1002/0471461296

[5] J. Warnatz, *Combustion Physical and Chemical Fundamentals, Modeling and Simulation, Experiments, Pollutant Formation*, 4th ed. Berlin, Heidelberg: Springer Berlin Heidelberg, 2006.

[6] C. Greenshields and H. Weller, *Notes on Computational Fluid Dynamics: General Principles*. CFD Direct Ltd, 2022. [Online]. Available: https://doc.cfd.direct/notes/cfd-general-principles/

# Study questions

1. What is the default diffusion model in OpenFOAM?
2. How do you change the used model?
3. What thermodynamic properties are calculated here in Cantera?
4. Which diffusion model `FickianFourier` represents?
5. What is the used closure in species and energy equations and how it is implemented?